CENTRO UNIVERSITÁRIO REDENTOR
ENGENHARIA DA COMPUTAÇÃO

IGOR DUARTE DE OLIVEIRA ALMEIDA

IMPLEMENTAÇÃO DO PROTOCOLO SELECTIVE REPEAT ARQ EM PYTHON

Itaperuna-RJ
2026

1. INTRODUÇÃO

Para que uma transmissão de dados em redes de computadores seja confiável, são necessários mecanismos capazes de detectar e corrigir erros. Em ambientes reais, os meios físicos de transmissão estão sujeitos a ruídos, atenuação e interferências que podem provocar a corrupção ou a perda de quadros durante o trânsito entre emissor e receptor. O controle de erros acontece na camada de enlace de dados e ocorre de maneira que o transmissor espera por uma confirmação positiva antes de passar para o próximo item de dados, caso contrário, quando o erro é detectado, os *frames* são retransmitidos. Esse processo é conhecido como ARQ (*Automatic Repeat reQuest*) (TANENBAUM; WETHERALL, 2011).

Os protocolos ARQ operam com base em três elementos: detecção de erros por parte do receptor, envio de confirmações positivas (ACK) para quadros corretamente recebidos, e retransmissão seletiva ou em massa dos quadros problemáticos pelo emissor. De acordo com Forouzan (2007), existem três protocolos principais que implementam o ARQ, sendo eles: o Stop-and-Wait ARQ, o Go-Back-N ARQ e o Selective Repeat ARQ.

O Stop-and-Wait ARQ, apesar de sua simplicidade de implementação, apresenta limitações de desempenho. O transmissor envia um único quadro e aguarda o respectivo ACK antes de prosseguir. Este processo não utiliza toda largura de banda disponível, pois só envia um *frame* por vez, além de o canal ficar ocioso enquanto espera o ACK do receptor.

O Go-Back-N ARQ em comparação ao Stop-and-Wait ARQ, permite que múltiplos quadros sejam enviados sem confirmação prévia, por meio de uma janela deslizante no transmissor. Entretanto, em caso de erro, todos os quadros anteriores ao incorreto devem ser retransmitidos, mesmo que tenham sido corretamente recebidos pelo destino. Essa estratégia desperdiça largura de banda quando a taxa de erros do canal é significativa.

Por fim, o Selective Repeat ARQ apresenta uma maior eficiência entre os três protocolos. Nessa abordagem, o receptor é capaz de aceitar e armazenar em *buffer* os quadros que chegam fora de ordem, enquanto envia confirmações negativas (NACK) especificando apenas os quadros corrompidos ou perdidos. O transmissor, por sua vez, retransmite exclusivamente esses quadros problemáticos, sem regredir desnecessariamente no fluxo de dados. Esse comportamento minimiza o número de retransmissões e maximiza a utilização da capacidade do canal.

O presente artigo tem como objetivo apresentar uma análise do protocolo Selective Repeat ARQ, abordando seus fundamentos teóricos, descrevendo seu modo de funcionamento e realizando uma implementação simples em Python.

O código desenvolvido para este trabalho está disponível no GitHub. O link a seguir acessa o repositório: https://github.com/IgorDuart3/codigo_python_selective_repeat_arq.git

2. FUNDAMENTAÇÃO TEÓRICA

2.1 A Camada de Enlace de Dados e os Controle de Fluxo e Controle de Erros

No modelo teórico OSI, as funções de comunicação são distribuídas em sete camadas hierárquicas. A camada de enlace de dados, segunda camada, é responsável pela comunicação confiável entre dois nós diretamente conectados. Suas principais responsabilidades são o controle de fluxo e o controle de erros, denominados coletivamente de controle de enlace de dados.

De acordo com Forouzan (2007), o controle de fluxo é um conjunto de procedimentos usados para controlar a quantidade de dados que o emissor pode enviar antes de receber confirmação dos dados transmitidos. O controle de fluxo é importante para prevenir que o buffer do receptor fique cheio evitando sobrecargas. Já o controle de erros trata do problema de quadros que chegam corrompidos ou simplesmente não chegam ao destino, sendo implementado na camada de enlace por meio de técnicas ARQ. Ambos os controles estão intimamente relacionados: o mecanismo ARQ depende diretamente de uma janela de transmissão para operar com eficiência, unindo assim o controle de fluxo e o controle de erros em um único mecanismo integrado.

2.2 Controle de Erros: ACK, NAK e Temporizadores

O termo controle de erros, na camada de enlace, refere-se tanto à detecção quanto à correção de erros. Ele é fundamentado em três elementos básicos. O primeiro é a detecção de erros: o receptor utiliza técnicas como CRC (Cyclic Redundancy Check) para verificar a integridade de cada quadro recebido. O segundo é o reconhecimento positivo, ACK (*Acknowledgment*). Sempre que um quadro é recebido corretamente, o receptor envia uma confirmação ao emissor. O terceiro é o reconhecimento negativo, NAK/NACK (*Negative Acknowledgment*). Quando o receptor detecta um quadro danificado ou percebe a ausência de um quadro esperado, envia ao transmissor um pedido explícito de retransmissão (FOROUZAN, 2007).

Além dos ACKs e NAKs, o controle de erros utiliza temporizadores para tratar a perda silenciosa de quadros ou de ACKs. O transmissor associa um temporizador a cada quadro enviado: se o ACK correspondente não chegar antes que o tempo limite seja atingido, o quadro é retransmitido. No Selective Repeat ARQ, cada quadro da janela de transmissão possui seu próprio temporizador independente, o que torna o controle mais preciso, porém exige maior complexidade de gerenciamento em comparação ao Stop-and-Wait ARQ, que mantém apenas um temporizador ativo por vez (TANENBAUM; WETHERALL, 2011).

2.3 Janela Deslizante e Pipelining

O principal gargalo do protocolo Stop-and-Wait é a ociosidade do canal durante o tempo em que o transmissor aguarda o ACK de cada quadro enviado. Para eliminar esse desperdício de tempo, os protocolos de janela deslizante introduzem o conceito de *pipelining*, em que o transmissor pode enviar múltiplos quadros consecutivos sem aguardar confirmação individual de cada um, até o limite definido pelo tamanho da janela de transmissão, $W \leq M/2$, em que $\mathrm{M} = 2^m$ e m é o tamanho, em bits, do campo número de sequência. Cada quadro recebe um número de sequência único, e o transmissor mantém uma lista dos quadros enviados, mas ainda não confirmados. À medida que os ACKs chegam, a janela desliza para frente, liberando espaço para novos quadros (STALLINGS, 2014).

2.4 Buffers e Numeração de Sequência

O conceito de buffer, na camada de enlace, é um mecanismo utilizado para armazenar temporariamente os quadros da janela que aguardam o ACK. Inclusive, essa é a principal diferença entre o Go-Back-N e o Selective Repeat ARQ. No Go-Back-N, o receptor não armazena quadros fora de ordem. No Selective Repeat, o receptor mantém um buffer de recepção capaz de armazenar até W quadros fora de ordem, onde W é o tamanho da janela do receptor (TANENBAUM; WETHERALL, 2011).

Quando um quadro com erro é identificado, o receptor envia um NAK com o número de sequência específico para o emissor e continua aceitando os quadros subsequentes que chegam corretamente, armazenando-os no buffer em suas posições correspondentes. Após a transmissão de todos os *frames*, o número de sequência é responsável por organizá-los, de maneira que eles sigam na ordem correta para a próxima camada.

3. DESENVOLVIMENTO

3.1 Funcionamento Detalhado do Selective Repeat ARQ

O Selective Repeat ARQ opera com duas janelas deslizantes independentes: a janela de transmissão, mantida pelo transmissor, e a janela de recepção, mantida pelo receptor.

A Figura 1 a seguir é um diagrama que ilustra como ocorre o processo de transmissão de dados entre emissor e receptor. Para haver uma interpretação clara da Figura 1 é necessário compreender os elementos que compõem a imagem. As janelas deslizantes estão representadas na Figura. A janela de transmissão está marcada de verde, e a janela de recepção marcada de vermelha. Os indicadores S_f e S_n são representações pertencentes à janela de transmissão. S_f indica o primeiro quadro da janela de transmissão aguardando por

seu ACK. S_n indica o próximo *frame* a ser enviado. R_n pertence exclusivamente à janela de recepção e indica o próximo *frame* esperado pelo receptor. Os relógios indicam os *timers* de cada quadro. Eles são ativados assim que o emissor envia o *frame* ao receptor.

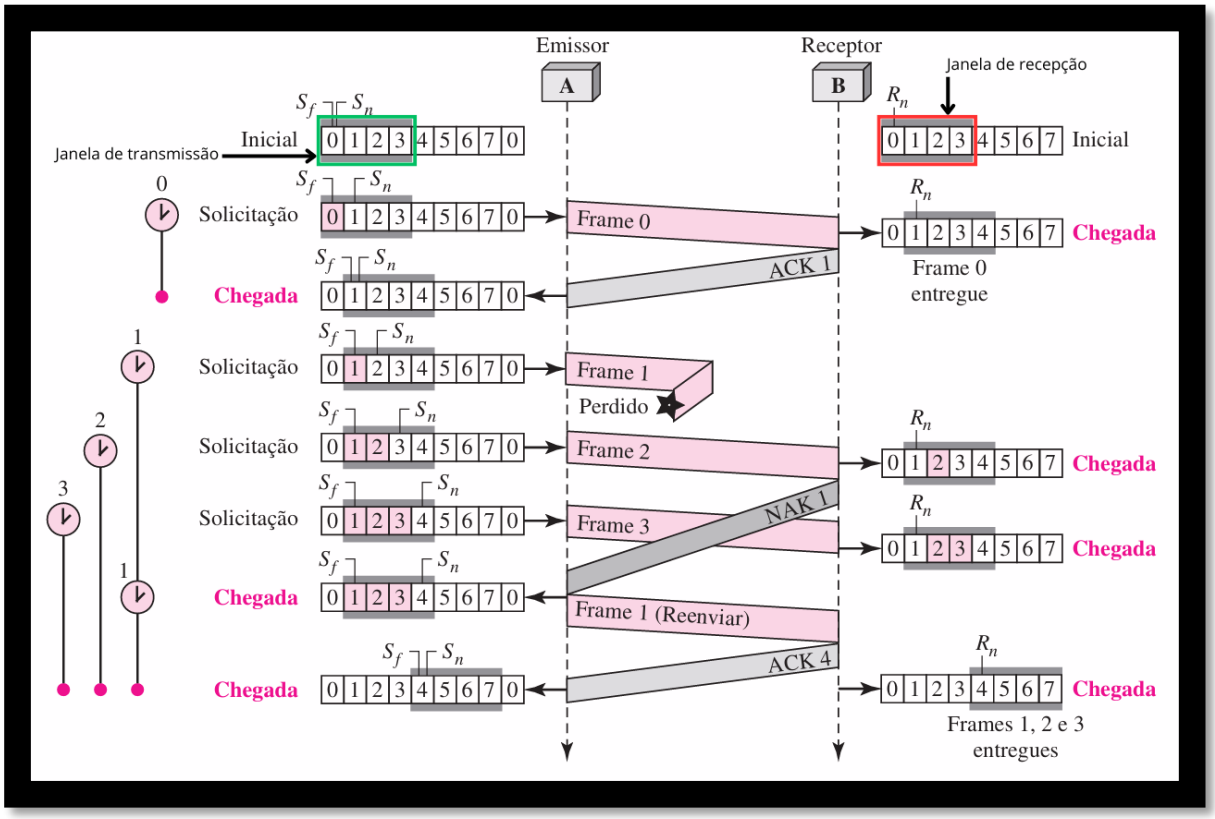

Figura 1: Diagrama de Fluxo de Dados.

Fonte: (FOROUZAN, 2007)

O diagrama ilustrado na Figura 1 representa o seguinte fluxo de dados: O emissor envia o quadro de número 0 (S_f) e o timer é ativado, mas logo é interrompido, pois o receptor já devolve o ACK rapidamente. Na próxima solicitação, S_f = 1, o frame enviado, o *timer* foi ativado, mas em seguida, o quadro foi perdido. O *timer* neste caso específico foi reiniciado quando o emissor recebeu o NAK e reenviou o quadro 1 novamente. Após a primeira solicitação do quadro em que S_f = 1, houveram duas novas solicitações sequencialmente, em que S_f = 2 e posteriormente S_f = 3. Ao final do envio do *frame* 3, o quadro 1 foi enviado novamente, como descrito anteriormente. Depois de recebê-lo, o ACK 4 foi enviado pelo receptor e liberou a movimentação da janela de transmissão. É importante reparar, que desde a falha do primeiro quadro 1, R_n se manteve em 1. Como os *frames* enviados pertenciam à janela de transmissão, no momento em que o receptor recebe o quadro 1 e já tem os quadros seguintes armazenados no *buffer*, ele libera 3 posições de uma só vez.

3.2 Implementação em Python

A implementação a seguir simula de maneira simples o protocolo Selective Repeat ARQ em Python. O programa, apresentado pela Figura 2, estrutura-se em uma única classe: *SelectiveRepeatSim*, que é responsável por toda lógica do que acontece em uma aplicação real do modelo de protocolo Selective Repeat ARQ.

simulacao_selective_repeat_arq.py > ...

```python
import random


class SelectiveRepeatSim:
    def __init__(self, total_pacotes, tamanho_janela, prob_perda):
        self.total_pacotes = total_pacotes
        self.tamanho_janela = tamanho_janela
        self.prob_perda = prob_perda
        self.pacotes_confirmados = [False] * total_pacotes
        self.tentativas_totais = 0

    def enviar(self):
        base = 0

        while base < self.total_pacotes:
            for i in range(base, min(base + self.tamanho_janela, self.total_pacotes)):
                if not self.pacotes_confirmados[i]:
                    self.tentativas_totais += 1
                    print(f"Transmitindo Pacote {i}...", end=" ")

                    if random.random() > self.prob_perda:
                        print("Recibo (ACK) recebido!")
                        self.pacotes_confirmados[i] = True
                    else:
                        print("FALHA (Pacote perdido/corrompido)")

            while base < self.total_pacotes and self.pacotes_confirmados[base]:
                base += 1
                print(f"Janela deslizou para a posição {base}")

            print("-" * 30)

        print(f"\nSucesso! Todos os {self.total_pacotes} pacotes entregues.")
        print(f"Total de transmissões realizadas: {self.tentativas_totais}")


if __name__ == "__main__":
    # Configuração: 10 pacotes, janela de tamanho 4, 30% de chance de perda
    simulacao = SelectiveRepeatSim(total_pacotes=10, tamanho_janela=4, prob_perda=0.3)
    print(f"Transmitindo {simulacao.total_pacotes} pacotes\n")
    print(f"Janela inicia na posição 0")
    simulacao.enviar()

```

Figura 2: Implementação do Selective Repeat ARQ em Python.

Fonte: Elaborado pelo autor

O código inicia importando a biblioteca *random*, que fará a simulação das falhas em um canal real. Logo em seguida, cria-se a classe *SelectiveRepeatSim*, que possui além do método *__init__* para iniciar o objeto, possui também o método enviar, que simula todo o processo de envio do *frame*.

No método *__init__* são definidos os atributos do sistema. São eles: a quantidade total de pacotes a serem transmitidos, o tamanho da janela deslizante (W), e a probabilidade de erro do canal. Para gerenciar o status de cada unidade de dados, pacotes_confirmados representa uma lista de booleanos que atua como o registro de reconhecimentos (ACKs) recebidos pelo emissor. Esse mecanismo permite que o sistema identifique rapidamente quais lacunas precisam ser preenchidas sem a necessidade de retransmitir o lote completo.

Dentro de cada iteração, o programa percorre apenas os índices que pertencem à janela atual, da base até o limite definido pelo tamanho da janela. Para cada quadro nesse intervalo que ainda não possui uma confirmação positiva, o sistema realiza uma tentativa de transmissão. A perda de pacotes é simulada através da comparação entre números gerados aleatoriamente pela biblioteca *random* e a probabilidade de erro definida inicialmente.

Seguindo com o código, a variável base só é incrementada quando o pacote mais antigo da sequência é confirmado com sucesso. Caso pacotes posteriores sejam recebidos, eles são marcados como confirmados no *buffer*, mas a janela não avança até que o pacote que falta seja corrigido e retorne o ACK. Essa etapa garante que os pacotes sejam entregues em ordem para a camada superior.

Após descrever a lógica construída, para executar o código, criamos o objeto simulação e chamamos o método enviar.

4. RESULTADOS

Para simular o funcionamento do protocolo Selective Repeat ARQ, utilizamos um exemplo fictício em que serão enviados 10 pacotes, a janela terá tamanho 4, respeitando a regra $W \leq M/2$ e a probabilidade de perda do canal em 30%.

Ao executar o programa, o resultado obtido está ilustrado pela Figura 3.

Ao observar a Figura 3, percebe-se que a janela de transmissão inicia na posição 0. As transmissões dos dois primeiros pacotes, 0 e 1 ocorreram perfeitamente. Mas ao transmitir os pacotes 2 e 3 houve alguma falha. Como os pacotes 0 e 1 são os primeiros da janela de transição, a janela deslizou para a posição 2.

O emissor dá inicio ao envio dos pacotes 2, 3, 4 e 5. Desta vez, os pacotes 2 e 4 foram transmitidos com sucesso, mas os pacotes 3 e 5 foram perdidos. A janela avançou da posição 2 para a posição 3, o que indica que o pacote 3 precisa ser reenviado. Como o pacote 4 foi

recebido com sucesso, e a janela tem tamanho 4, o emissor precisa enviar os pacotes 3, 5 e 6, e todos eles foram corrompidos. Portanto a janela não avança nenhuma posição.

Enviando novamente os pacotes 3, 5 e 6, desta vez o receptor recebeu-os corretamente, permitindo que a janela avançasse para as próximas casas. Então, os próximos pacotes a serem transmitidos serão os pacotes 7, 8 e 9, completando a sequência dos 10 bits.

No final, foram necessárias 17 transmissões para que todos os pacotes fossem transferidos.

```
Transmitindo 10 pacotes

Janela inicia na posição 0
Transmitindo Pacote 0... Recibo (ACK) recebido!
Transmitindo Pacote 1... Recibo (ACK) recebido!
Transmitindo Pacote 2... FALHA (Pacote perdido/corrompido)
Transmitindo Pacote 3... FALHA (Pacote perdido/corrompido)
Janela deslizou para a posição 1
Janela deslizou para a posição 2
------------------------------
Transmitindo Pacote 2... Recibo (ACK) recebido!
Transmitindo Pacote 3... FALHA (Pacote perdido/corrompido)
Transmitindo Pacote 4... Recibo (ACK) recebido!
Transmitindo Pacote 5... FALHA (Pacote perdido/corrompido)
Janela deslizou para a posição 3
------------------------------
Transmitindo Pacote 3... FALHA (Pacote perdido/corrompido)
Transmitindo Pacote 5... FALHA (Pacote perdido/corrompido)
Transmitindo Pacote 6... FALHA (Pacote perdido/corrompido)
------------------------------
Transmitindo Pacote 3... Recibo (ACK) recebido!
Transmitindo Pacote 5... Recibo (ACK) recebido!
Transmitindo Pacote 6... Recibo (ACK) recebido!
Janela deslizou para a posição 4
Janela deslizou para a posição 5
Janela deslizou para a posição 6
Janela deslizou para a posição 7
------------------------------
Transmitindo Pacote 7... Recibo (ACK) recebido!
Transmitindo Pacote 8... Recibo (ACK) recebido!
Transmitindo Pacote 9... Recibo (ACK) recebido!
Janela deslizou para a posição 8
Janela deslizou para a posição 9
Janela deslizou para a posição 10
------------------------------

Sucesso! Todos os 10 pacotes entregues.
Total de transmissões realizadas: 17
```

Figura 3: Resultado da execução do programa.

Fonte: Elaborado pelo autor

5. CONCLUSÃO

Após a verificação dos resultados é possível perceber que o programa funciona corretamente. Para futuros trabalhos, é indicado que o programa seja melhor desenvolvido, criando novas classe, para que o modelo fique completo. Seria necessário separar o programa em 3 classes. A primeira para lidar exclusivamente com o canal, a segunda para lidar exclusivamente com o transmissor e a outra com o receptor.

Entretanto, a implementação do algoritmo do protocolo Selective Repeat ARQ funcionou como esperado, representando o funcionamento desta tecnologia.

Referências

FOROUZAN, B. A. Transmissão de Dados e Redes de Computadores. 4. ed. New York: McGraw-Hill, 2007.

STALLINGS, W. Data and Computer Communications. 10. ed. Upper Saddle River: Pearson, 2014.

TANENBAUM, A. S.; WETHERALL, D. J. Redes de Computadores. 5. ed. Upper Saddle River: Pearson Prentice Hall, 2011.